TP 4 — Fédération de données multi-SGBD avec PySpark

Durée : 2 h 30 — **Séance 4 — après-midi**

Au TP 3, Spark a joint deux tables de deux bases **MySQL**. Aujourd'hui, vous joignez **quatre tables** hébergées dans **quatre SGBD différents** — comme si elles vivaient dans une seule base. Aucun SGBD ne sait faire cela seul ; Spark, si.

Matériel fourni : `TP4_federation_multi_sgbd.ipynb` • `init_tp4_mysql.sql` • `init_tp4_postgres.sql` • `init_tp4_oracle.sql` • `init_tp4_sqlserver.sql`

Livrable (à pousser sur GitHub avant la séance 5) : le notebook exécuté de bout en bout **avec ses sorties** + `docs/AUDIT_TP4.md`.

Le scénario

Notre plateforme e-commerce sénégalaise a grandi : chaque équipe a choisi son SGBD, et il n'existe **aucune clé étrangère** entre les systèmes. La direction demande pourtant un chiffre d'affaires consolidé par ville, par catégorie de produit et par moyen de paiement.

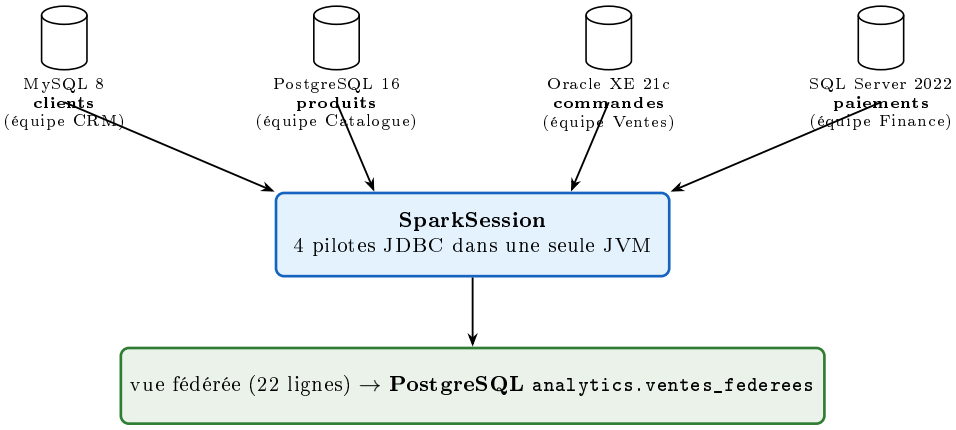


Table	SGBD	Port	Base / schéma	Lignes
clients	MySQL 8	3306	`ecommerce_crm`	12
produits	PostgreSQL 16	5432	`ecommerce_catalogue`	10
commandes	Oracle XE 21c	1521	schéma `SPARK_USER`	30
paiements	SQL Server Express	1433	`ecommerce_paiements`	27

Compte technique commun (créé par les scripts, moindre privilège) : `spark_user` / `Ucad2026!`.

Déroulé de la séance (2 h 30)

Partie	Contenu	Durée
A	Mise en place des 4 SGBD (scripts `init_tp4_*.sql`)	30 min
B	Les 4 connecteurs JDBC et la SparkSession	15 min
C	Lire les 4 tables (exercices 1–3)	25 min
D	La vue fédérée : jointures multi-SGBD (exercices 4–6)	30 min
E	Audit d'intégrité, analyses, entrepôt (exercices 7–9)	35 min
F	Discussion guidée et quiz éclair	15 min

Partie A — Mise en place des quatre SGBD

prérequis · 30 min

Exécutez chaque script avec le **compte administrateur** du SGBD concerné. Chaque script crée la base (ou le schéma), le compte `spark_user` et les données ; tous sont **rejouables**.

```
mysql -u root -p < init_tp4_mysql.sql
psql -U postgres -h localhost -f init_tp4_postgres.sql
sqlplus system/MdpSystem@//localhost:1521/XEPDB1 @init_tp4_oracle.sql
sqlcmd -S localhost -U sa -P MdpSa -i init_tp4_sqlserver.sql
```

(Équivalents graphiques acceptés : Workbench, pgAdmin, SQL Developer, SSMS.) Vérifiez le décompte affiché en fin de script : 12 / 10 / 30 / 27. La section 2 du notebook testera ensuite les 4 ports — consultez l'annexe « pièges connus » en cas d'échec (TCP/IP de SQL Server notamment).

Partie B — Quatre pilotes, une session

sections 3–4 · 15 min

Le notebook télécharge les 4 pilotes JDBC (`mysql-connector-j`, `postgresql`, `ojdbc11`, `mssql-jdbc`) dans `jars/`, puis crée la SparkSession avec `spark.jars` = les 4 chemins séparés par des virgules. Salle sans Internet : copiez les jars fournis par l'enseignant dans `jars/` avant d'exécuter la cellule. Rappel du TP 3 : `spark.jars` se fixe **à la création** de la session.

Partie C — Lire les quatre tables

exercices 1–3 · 25 min

La lecture MySQL est guidée ; à vous PostgreSQL (ex. 1), Oracle (ex. 2) et SQL Server (ex. 3) via la fonction `lire_jdbc`. Observez les **dialectes** : Oracle renvoie des colonnes en MAJUSCULES et des `NUMBER` → `decimal(38,10)` à normaliser ; la table SQL Server se nomme `dbo.paiements`, celle d'Oracle `spark_user.commandes`. Relevez les 4 comptages dans `RELEVES`.

Partie D — La vue fédérée

exercices 4–6 · 30 min

Créez les 4 vues temporaires (ex. 4) puis étendez la jointure de 2 à 3 puis à **4 SGBD** (ex. 5–6) : la table finale `ventes_federees` garde les commandes *livrées* dont le paiement est *réussi*. Expliquez dans la cellule de réflexion pourquoi les comptages passent de 30 à 29, 28, puis 22 lignes.

Partie E — Audit, analyses, entrepôt

exercices 7–9 · 35 min

Audit (ex. 7–8) : aucune clé étrangère ne traverse les SGBD — à vous de vérifier l'intégrité par **anti-jointures** : commandes sans client, commandes sans produit, paiements orphelins, livrées impayées. **Analyses** (ex. 9) : CA fédéré total, par ville, par catégorie, par moyen de paiement. **Sous le capot** : `explain()` sur un filtre Oracle — repérez `PushedFilters`. **Entrepôt** : écrivez la vue fédérée dans PostgreSQL (`analytics.ventes_federees`) puis relisez-la pour preuve.

Partie F — Discussion guidée et quiz

en groupes | 15 min

Discussion (groupes de 3–4, restitution 1 minute) :

1. La vue fédérée interroge les 4 bases de *production* à chaque exécution : quels risques, et quelle architecture du cours (séances 6–7) y répond ?
2. Qui, dans l'entreprise, devrait être responsable du paiement orphelin `PAY27` découvert par l'audit ?
3. Citez un cas sénégalais réel où la fédération multi-SGBD serait nécessaire (administrations, banques/fintech, opérateurs).

Quiz éclair (auto-évaluation) :

1. Pourquoi faut-il quatre `.jar` différents ?
2. Que se passe-t-il si l'option `driver` est omise ?
3. Pourquoi la jointure interne perd-elle `CMD21` et `CMD22` ?
4. Qu'est-ce que le *predicate pushdown* et qui exécute le filtre ?
5. `spark.jars` peut-il changer après création de la session ?

Le livrable, en détail

À pousser sur votre dépôt GitHub **avant la séance 5** :

1. `notebooks/TP4_federation_multi_sgbd.ipynb` complété et exécuté de bout en bout, **avec ses sorties** ;
2. `docs/AUDIT_TP4.md` : les trois anomalies d'intégrité (identifiants, table et SGBD source, impact métier en une phrase chacune) + le tableau de relevés recopié.

Critère	Poids
Notebook complet, exécuté sans erreur, relevés remplis	50 %
`AUDIT_TP4.md` : anomalies correctes et impact argumenté	30 %
Propreté du dépôt Git (structure, README, messages de commit)	20 %

Avant de partir — checklist

☐ Les 4 scripts `init_tp4_*.sql` s'exécutent sans erreur
☐ La cellule de pré-vérification affiche 4 `[ OK ]`
☐ La vue fédérée compte 22 lignes ; CA total 1 452 000 FCFA
☐ Les 3 anomalies d'audit sont identifiées et documentées
☐ `analytics.ventes_federees` est visible dans pgAdmin/`psql`
☐ Notebook + `AUDIT_TP4.md` poussés sur GitHub

Séance 5 : jointures avancées et stratégies d'exécution (broadcast, shuffle) — relisez d'ici là la partie 9 du notebook (*pushdown*).

Annexe — pièges connus (à lire en cas de blocage)

SQL Server Express : TCP/IP est **désactivé par défaut**. SQL Server Configuration Manager → Protocoles pour SQLEXPRESS → TCP/IP → Activer ; onglet Adresses IP, section IPAll : Port TCP = 1433 et champ « Ports dynamiques TCP » vidé ; redémarrer le service. Vérifiez aussi que l'instance accepte l'**authentification en mode mixte** (sinon le login `spark_user` sera refusé).

Oracle XE : démarrer le *listener* (`OracleOraDB21Home1TNSListener`) **avant** le service `OracleServiceXE` ; le service de la base pluggable est `XEPDB1`, pas `XE`. La table est `spark_user.commandes` (schéma = utilisateur en Oracle).

MySQL : conservez le suffixe d'URL `?useSSL=false&allowPublicKeyRetrieval=true` (TP 3).

Tous SGBD : l'option `driver` est **obligatoire** dans chaque lecture — sans elle, erreur « No suitable driver ».

Pare-feu Windows : si un port répond `[ECHEC]` alors que le service tourne, autorisez le port dans le pare-feu ou testez avec `Test-NetConnection localhost -Port 1433` (PowerShell).